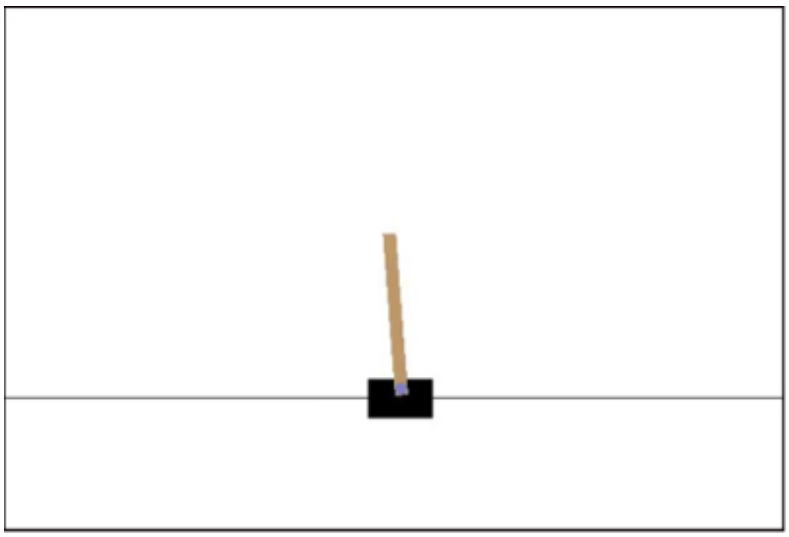


We can represent an agent as a neural network that approximates the policy function—it accepts a state and outputs an action, or more typically a probability distribution over actions. The following listing shows an example of a three-layer network.

Listing 6.5. Defining an agent

```
import numpy as np
import torch

def model(x,unpacked_params):
    l1,b1,l2,b2,l3,b3 = unpacked_params          1
    y = torch.nn.functional.linear(x,l1,b1)      2
    y = torch.relu(y)                            3
    y = torch.nn.functional.linear(y,l2,b2)
    y = torch.relu(y)
    y = torch.nn.functional.linear(y,l3,b3)
    y = torch.log_softmax(y,dim=0)               4
    return y
```

- ***1*** Unpacks the parameter vector into individual layer matrices
- ***2*** A simple linear layer with bias
- ***3*** A rectified linear unit activation function
- ***4*** The last layer will output log probabilities over actions.

The function in listing 6.5 defines a 3-layer neural network. The first two layers use rectified linear unit activation functions, and the last layer uses a log-softmax activation function so that we get log probabilities over actions as the final output. Notice that this function expects an input state, `x`, and